UNIBALSAS

Análise e Desenvolvimento de Sistemas (ADS)

RELATÓRIO TÉCNICO

TED 03 - GERENCIADOR DE PERSISTÊNCIA DE DADOS EM ARQUIVOS ESTRUTURADOS (CSV E JSON)

Katson Kawan Cardoso de Carvalho

Bonifácio Pinto Costa Neto

Phelype Rocha Marques

Disciplina: ALGORITMOS E PROGRAMAÇÃO AVANÇADA

Balsas - MA, 2026

1. Identificação

Item	Informação
Instituição	UniBalsas
Curso	Análise e Desenvolvimento de Sistemas (ADS)
Disciplina	ALGORITMOS E PROGRAMAÇÃO AVANÇADA
Integrantes	Katson Kawan Cardoso de Carvalho; Bonifácio Pinto Costa Neto; Phelype Rocha Marques
Atividade	TED 03 - Persistência de dados CSV e JSON
Entrega	31/10/2026

2. Introdução e objetivos

A atividade propõe o desenvolvimento de um gerenciador de dados estruturados capaz de ler, converter e atualizar arquivos CSV (valores separados por vírgulas) e JSON (JavaScript Object Notation). Os arquivos permitem armazenar registros fora da memória da aplicação, viabilizando a sua recuperação em execuções posteriores.

O objetivo geral é implementar persistência confiável em Python. Os objetivos específicos compreendem preservar registros na conversão entre formatos, empregar gerenciadores de contexto na manipulação de arquivos, tratar falhas de entrada/saída e proteger versões anteriores com backups automáticos.

3. Descrição da solução

Foi implementada uma aplicação de linha de comando (CLI), sem dependências externas para executar suas funcionalidades. A interface centraliza três comandos: **ler**, **converter** e **atualizar**. A organização modular distribui as responsabilidades entre os componentes apresentados a seguir.

Módulo	Responsabilidade
io_csv.py / io_json.py	Leitura e escrita de arquivos com with open e UTF-8.
validacao.py	Verificação de campos, tipos textuais e consistência dos registros.
conversao.py	Controle do sentido da conversão e do caminho de destino.
backup.py	Cópia binária dos dados antes da sobrescrita.
persistencia.py	Atualização, gravação temporária e substituição segura.
main.py	Comandos e mensagens claras para o usuário.

4. Implementação

4.1. Leitura de arquivos CSV e JSON

A leitura do CSV utiliza **csv.reader** com **newline=""**, necessário para tratamento adequado de registros com quebras de linha internas. A codificação **utf-8-sig** reconhece UTF-8 e aceita arquivos com marca de ordem de bytes (BOM).

```
with open(caminho, "r", encoding="utf-8-sig", newline="") as arquivo:
    leitor = csv.reader(arquivo, strict=True)
    # Validação do cabeçalho e de cada linha lida.
    ...
```

O módulo JSON utiliza **json.load** e verifica chaves repetidas. Os dois leitores retornam a mesma representação interna: uma lista de dicionários cujos valores são textos.

```
with open(caminho, "r", encoding="utf-8-sig") as arquivo:
    registros = json.load(
        arquivo, object_pairs_hook=_objeto_sem_chaves_duplicadas
    )
return validar_registros(registros)
```

4.2. Escrita e conversão sem perdas silenciosas

O fluxo de conversão consiste em ler a origem, validar os registros e gravá-los no formato de destino. Para preservar os acentos, o JSON é escrito com **ensure_ascii=False**. No CSV, o DictWriter preserva a sequência dos campos e realiza corretamente o escape de aspas, vírgulas e quebras de linha.

```
with open(caminho, "x", encoding="utf-8") as arquivo:
    json.dump(registros, arquivo, ensure_ascii=False, indent=2)
    arquivo.write("\n")
```

Há uma restrição intencional de integridade: o JSON de entrada deve conter lista não vazia de objetos, com campos uniformes e valores do tipo texto. Caso surjam números, booleanos, valores nulos, estruturas aninhadas ou campos irregulares, a aplicação informa a incompatibilidade, em vez de perder a informação de tipo sem aviso.

4.3. Atualização dos registros

O comando de atualização localiza um registro pela chave **id**, exige ocorrência única e confirma a existência do campo a alterar. Caso o novo valor seja igual ao atual, não realiza gravação nem cria backup desnecessário.

4.4. Backup automático antes da modificação

O mecanismo de backup cria, no diretório **backups/**, uma cópia binária do arquivo original com nome contendo data, hora e identificador único. O uso de **open(..., "xb")** impede que uma cópia de mesmo nome seja sobrescrita acidentalmente. O flush e o fsync são utilizados para concluir a escrita da cópia antes de prosseguir.

```
with open(origem, "rb") as entrada, open(copia, "xb") as saida:
    shutil.copyfileobj(entrada, saida)
    saida.flush()
    os.fsync(saida.fileno())
```

Na função de persistência, o backup é criado **antes** da geração e da substituição do arquivo definitivo. Se a cópia não puder ser criada, uma exceção cancela o processo, deixando o arquivo anterior inalterado.

```
copia = criar_backup(destino) if destino.exists() else None
temporario = destino.with_name(
    f".{destino.name}.{uuid4().hex}.tmp"
)
try:
    ESCRITORES[extensao](temporario, registros)
    os.replace(temporario, destino)
finally:
    temporario.unlink(missing_ok=True)
```

4.5. Tratamento de exceções

A interface captura **FileNotFoundError**, **PermissionError**, **UnicodeError**, **JSONDecodeError**, **csv.Error**, falhas gerais de **OSError** e erros próprios de validação. As falhas resultam em mensagens objetivas e código de saída 1, sem exibir rastreamentos internos ao usuário.

```
except FileNotFoundError as exc:
    print(f"[ERRO] Arquivo ou diretório não encontrado: {exc}",
          file=sys.stderr)
except PermissionError as exc:
    print(f"[ERRO] Permissão negada: {exc}", file=sys.stderr)
```

A abertura dos arquivos é realizada com **with open(...)**; assim, o fechamento é automático também quando uma exceção ocorre durante a leitura ou a escrita. O uso de arquivo temporário e **os.replace** reduz a exposição a arquivos finais parcialmente gravados.

5. Testes e evidências de execução

5.1. Estratégia de testes automatizados

Foi desenvolvida uma suíte de testes com **unittest**, executada em diretórios temporários, sem modificar os arquivos originais de exemplo. Os testes confrontam os registros convertidos com os registros de origem e verificam o conteúdo do backup byte a byte.

Grupo de testes	Verificação realizada
Leitura e conversão	Leitura CSV/JSON; conversão nos dois sentidos; ida e volta.
Backup e atualização	Cópia anterior preservada; atualização CSV/JSON; valor idêntico.
Entradas inválidas	Arquivo ausente; UTF-8 inválido; JSON malformado; CSV irregular.
Integridade	Tipos JSON não textuais; chaves e IDs duplicados.
Falhas simuladas	Falha no backup e falha na substituição final preservam o original.

5.2. Resultado observado

Na execução registrada, **18 testes foram realizados e todos passaram**. O trecho de erro exibido durante o teste de arquivo ausente é intencional: faz parte da verificação da mensagem apresentada pela aplicação, não indicando falha na suíte.

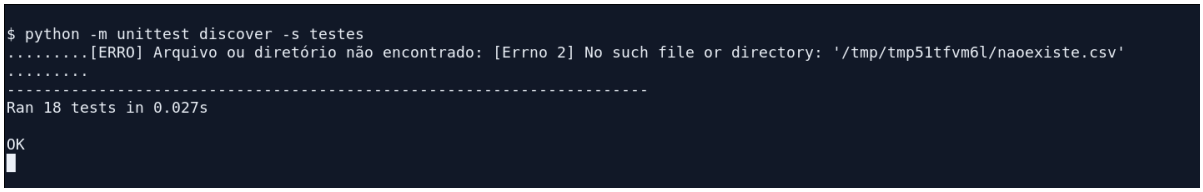


Figura 1 - Execução da suíte unittest: 18 testes, resultado OK. Fonte: execução do projeto.

Além dos testes automatizados, foram executados quatro cenários diretamente pelo terminal. As próximas figuras apresentam os comandos, os arquivos produzidos e as mensagens observadas, permitindo verificar os requisitos da TED 03.

5.3. Evidência 1 - Leitura CSV e conversão para JSON

O arquivo **dados/clientes.csv**, contendo quatro registros, foi convertido com o comando **python main.py converter dados/clientes.csv resultados/clientes_convertidos.json**. A aplicação identificou quatro registros e gravou o arquivo de saída sem erro.

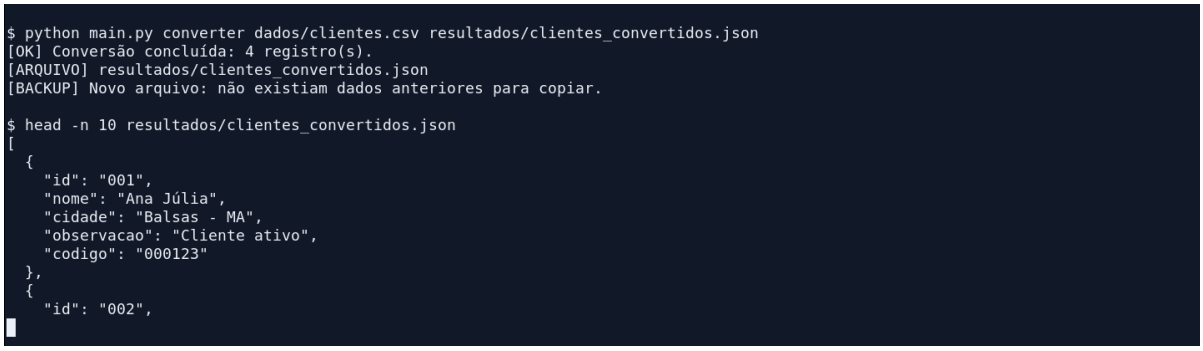


Figura 2 - Conversão CSV para JSON, seguida da visualização do conteúdo gerado. Fonte: execução do projeto.

Análise do resultado

O arquivo JSON apresenta os mesmos campos do CSV (id, nome, cidade, observacao e codigo) e preserva caracteres acentuados, aspas em observações e zeros à esquerda no código, pois os valores são tratados como texto. A mensagem de backup informa corretamente que não havia versão anterior do arquivo recém-criado.

A leitura foi validada adicionalmente pela suíte automatizada. Um teste de ida e volta

(CSV → JSON → CSV) confrontou o conteúdo final com a estrutura de registros original, sem diferenças nos campos ou valores.

5.4. Evidência 2 - Leitura JSON e conversão para CSV

O arquivo **dados/clientes.json** foi convertido para **resultados/clientes_convertidos.csv**. A saída do terminal indica a gravação de quatro registros e a ausência de backup na criação do novo arquivo.

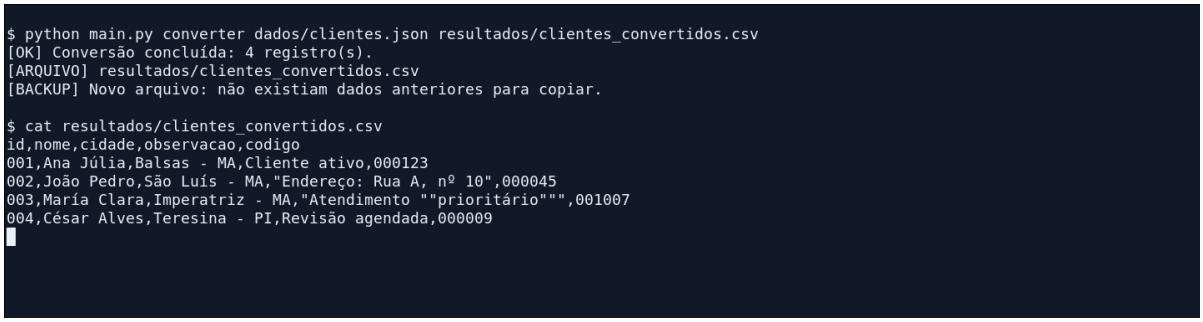


Figura 3 - Conversão JSON para CSV e exibição do arquivo CSV gerado. Fonte: execução do projeto.

Análise do resultado

O cabeçalho do CSV apresenta os cinco campos de origem. Os registros contêm acentos e códigos como “000123”, mantidos sem truncamento. Valores que possuem vírgulas ou aspas são devidamente delimitados e escapados pelas rotinas do módulo csv.

Essa verificação demonstra a compatibilidade entre os arquivos de exemplo dos dois formatos. Os testes automatizados abrangem ainda quebras de linha no interior de campos, tratamento que exige o uso adequado de **newline=""** no módulo CSV.

5.5. Evidência 3 - Atualização com backup anterior

No arquivo CSV previamente convertido, foi executada a atualização do registro de **id 002**, modificando o campo **cidade** para “Imperatriz - MA”. Antes de substituir o CSV, o sistema gerou uma cópia de segurança com extensão **.bak**.



Figura 4 - Atualização, identificação do arquivo de backup e conferência do registro modificado. Fonte: execução do projeto.

Análise do resultado

A saída informa o caminho da cópia anterior em **resultados/backups/**. Em seguida, o terminal lista o arquivo de backup e exibe a linha cujo identificador é 002, comprovando que o novo valor foi persistido. O backup retém o conteúdo existente antes da alteração.

A ordem das operações é garantida pelo código: criar backup, escrever temporário e substituir o destino. A suíte também simula falha no backup e falha na troca de arquivos, verificando que o conteúdo anterior permanece íntegro em ambos os casos.

5.6. Evidência 4 - Tratamento de erro de arquivo

Foi solicitada a leitura de **dados/arquivo_inexistente.csv**, inexistente no projeto. Em vez de encerrar com um rastreamento de exceção, a aplicação mostrou a causa do problema e devolveu código de saída 1, conforme esperado para uma operação não concluída.

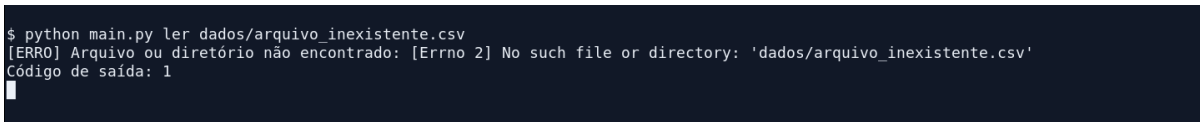


Figura 5 - Erro de arquivo inexistente e código de saída 1. Fonte: execução do projeto.

6. Conclusão

Os objetivos propostos foram atendidos nos cenários implementados e testados. A aplicação permite leitura, conversão e atualização de registros CSV/JSON, com

codificação UTF-8, gerenciadores de contexto, validação estrutural, exceções tratadas e backup prévio das versões existentes. As evidências de execução e os 18 testes automatizados sustentam os resultados observados.

Entre os desafios técnicos estiveram a preservação de conteúdo textual com aspas, acentos e zeros à esquerda e o controle da ordem entre backup e atualização. Para evitar conversões semanticamente incorretas, a solução rejeita explicitamente JSON com tipos que o CSV não consegue representar de modo reversível. Como aprimoramentos futuros, podem ser incluídos formatos mais complexos e controle de gravações simultâneas.

7. Referências

PYTHON SOFTWARE FOUNDATION. **csv - CSV File Reading and Writing**. Disponível em: https://docs.python.org/3/library/csv.html. Acesso em: 28 out. 2026.

PYTHON SOFTWARE FOUNDATION. **json - JSON encoder and decoder**. Disponível em: https://docs.python.org/3/library/json.html. Acesso em: 28 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Input and Output**. Disponível em: https://docs.python.org/3/tutorial/inputoutput.html. Acesso em: 28 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Built-in Exceptions**. Disponível em: https://docs.python.org/3/library/exceptions.html. Acesso em: 28 out. 2026.